# Q30: Inside PyTorch

torch.compile layers + autodiff memory

DLS 2026 — Kukla

October 2026

# The question

**Student question (#29)**
How does the PyTorch compiler work and what are its layers? Why does functional autodiff use far more memory than tape-based autodiff?

# Eager vs. compiled

- Default: **eager** — ops dispatch immediately, easy to debug, slow.
- `torch.compile`: JIT-compiles, keeps eager semantics, recovers speed.
- Philosophy: *capture what you can, fall back where you must*.

# The torch.compile stack

- **TorchDynamo**: trace bytecode $\rightarrow$ FX graph; *graph breaks* on untraceable code.
- **AOTAutograd**: capture forward *and* backward ahead of time.
- **TorchInductor**: fuse ops, lower to kernels.
- **Triton**: the emitted GPU kernel language.

# Two flavors of autodiff

- **Reverse-mode / tape**: record a tape forward, walk it backward (classic autograd).
- **Functional**: transform a pure fn into its gradient fn (`grad`, `vmap`, `jit`).

# Why functional AD can use more memory

- **Activations** saved for backward dominate training memory.
- A naive functional form may retain more intermediates than a tuned tape.
- Fix: **gradient checkpointing** — save few activations, *recompute* the rest.

Trade extra compute for a big memory cut (can turn OOM into trainable).